

Énoncé : Optimisation du Planning de Livraison (Livreur d'Élite)

Contexte

Tu incarnes un livreur indépendant à vélo. Plusieurs clients te proposent des courses pour la journée. Chaque course possède une **valeur** (ton pourboire), mais aussi une **deadline** (l'heure maximale, en minutes, à laquelle la course doit être terminée sous peine d'annulation).

Toutes les courses prennent exactement **1 minute** à être effectuées, et la journée commence à la minute 0. Tu ne peux faire qu'une seule course à la fois.

Objectif

L'objectif est de choisir et de planifier les courses de manière à **maximiser ton gain total**, tout en respectant strictement les deadlines de chaque course choisie.

Consignes de codage

Tu dois écrire une fonction `max_gain_livraisons(courses)` en Python.

- **courses** : Une liste de dictionnaires. Chaque dictionnaire représente une course : `{"id": str, "deadline": int, "valeur": int}`.
- **Valeur de retour** : Un **entier** représentant le gain maximal cumulé.

💡 **Le piège et l'astuce algorithmique (Pourquoi utiliser `heapq` ?)** : Si tu tries simplement les courses par valeur ou par deadline, tu vas te faire piéger. La stratégie gagnante :

1. Trie d'abord toutes tes courses par **deadline croissante**.
2. Crée un tas (`heapq`) pour stocker les **valeurs** des courses que tu acceptes provisoirement.
3. Parcours les courses triées. Pour chaque course, ajoute sa valeur dans ton tas.
4. Si le nombre de courses dans ton tas dépasse la deadline de la course actuelle (c'est-à-dire que tu n'as plus assez de temps physique pour tout livrer), **éjecte immédiatement la course la moins rentable de ton tas** (le minimum, accessible en $O(1)$ grâce au tas).

Exemple de comportement attendu

```
courses = [  
    {"id": "A", "deadline": 1, "valeur": 40},  
    {"id": "B", "deadline": 1, "valeur": 10},  
    {"id": "C", "deadline": 2, "valeur": 50},  
    {"id": "D", "deadline": 2, "valeur": 60}  
]  
  
# À la minute 1, les courses A et B expirent. On ne peut en faire qu'une. On  
# choisit A (40).  
# À la minute 2, les courses C et D expirent. On peut faire l'une d'elles à la  
# minute 1 (à la place de A) et l'autre à la minute 2.  
# Le choix optimal est de faire A à la minute 1 (40) ou C, et D à la minute 2  
# (60).
```

```
# Attends... Si on fait C (50) à la minute 1 et D (60) à la minute 2, on gagne 110
! C'est mieux que d'avoir pris A.

print(max_gain_livraisons(courses))
# Sortie attendue : 110
```

Prototype de la fonction

```
import heapq

def max_gain_livraisons(courses: list[dict]) -> int:
    """
    Calcule le gain maximum possible en choisissant les courses de manière
    optimale.

    :param courses: Liste de dictionnaires, ex: [{"id": "A", "deadline": 2,
    "valeur": 50}]
    :return: Gain total maximal (int)
    """
    # À toi de jouer
    pass
```

Suite de Tests

```
print("Démarrage des tests pour le Planificateur de Livraisons...")

# Test 1 : Aucune course disponible
print("Test 1...", max_gain_livraisons([]))
assert max_gain_livraisons([]) == 0, "Échec Test 1 : Sans course, le gain est de 0"

# Test 2 : Deadlines identiques, arbitrage nécessaire
courses_identiques = [
    {"id": "A", "deadline": 1, "valeur": 20},
    {"id": "B", "deadline": 1, "valeur": 100},
    {"id": "C", "deadline": 1, "valeur": 50}
]
print("Test 2...", max_gain_livraisons(courses_identiques))
assert max_gain_livraisons(courses_identiques) == 100, "Échec Test 2 : On doit juste prendre la plus rentable"

# Test 3 : Exemple complexe de l'énoncé (Changement d'avis en cours de route)
courses_arbitrage = [
    {"id": "A", "deadline": 1, "valeur": 40},
    {"id": "B", "deadline": 1, "valeur": 10},
```

```
{ "id": "C", "deadline": 2, "valeur": 50},
{ "id": "D", "deadline": 2, "valeur": 60}
]
print("Test 3...", max_gain_livraisons(courses_arbitrage))
assert max_gain_livraisons(courses_arbitrage) == 110, "Échec Test 3 : L'optimum
est 110 (50 + 60)"

# Test 4 : Grande deadline mais valeurs faibles vs petites deadlines valeurs
fortes
courses_melange = [
    { "id": "A", "deadline": 2, "valeur": 10},
    { "id": "B", "deadline": 2, "valeur": 15},
    { "id": "C", "deadline": 1, "valeur": 100},
    { "id": "D", "deadline": 3, "valeur": 5}
]
# On peut faire C (t=1), B (t=2), A (t=3) -> Total = 125
print("Test 4...", max_gain_livraisons(courses_melange))
assert max_gain_livraisons(courses_melange) == 125, "Échec Test 4 : L'optimum est
125"

# Test 5 : Test de performance (Grand nombre de données avec éjections massives du
tas)
import random
random.seed(42)
courses_perf = []
for i in range(5000):
    courses_perf.append({
        "id": f"C_{i}",
        "deadline": random.randint(1, 500), # Beaucoup de conflits (5000 courses
pour 500 slots max)
        "valeur": random.randint(10, 1000)
    })

print("Test 5 (Test de performance)...")
resultat = max_gain_livraisons(courses_perf)
print(f"Gain max trouvé sur 5000 courses : {resultat}")
# Si le code est en O(N^2), le test va ramer ou crash. Avec heapq, c'est < 0.05
seconde.
assert resultat > 0, "Échec Test 5"

print("\nTous les tests sont validés ! Tu as dompté le tas.")
```